SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

॥वसुधैव कुटुम्बकम्॥

PATTERN RECOGNITION

Computer Science and Engineering
Batch 2023-27

Name: Sukhada Bhoyar
PRN: 23070521033
Semester: VI
Section: A

Dimensionality Reduction using Kernel PCA

Abstract

The exponential growth of high-dimensional data in contemporary computational science necessitates advanced methodologies for dimensionality reduction to facilitate efficient processing, visualization, and classification. While Principal Component Analysis (PCA) remains the foundational linear technique for identifying directions of maximum variance, its inherent reliance on linear transformations limits its efficacy when dealing with complex, non-linear manifolds. This study investigates the extension of the classical PCA framework through Kernel Principal Component Analysis (Kernel PCA), which utilizes kernel functions to map input data into a high-dimensional Reproducing Kernel Hilbert Space (RKHS). In this enriched feature space, non-linear relationships in the original input space are transformed into linear structures, enabling the application of traditional linear decomposition. Through an experimental analysis involving non-linearly distributed data, this study demonstrates the superiority of Kernel PCA in producing a feature space where classes become linearly separable. Utilizing a Support Vector Machine (SVM) with a linear kernel as a diagnostic classifier, the results highlight a definitive improvement in classification performance and decision boundary clarity. The visualization of these boundaries confirms that Kernel PCA effectively "unfolds" complex data structures, overcoming the fundamental constraints of linear projection and significantly enhancing the robustness of feature representation.

Table of Contents

Dimensionality Reduction using Kernel PCA 2
- Abstract 2
- Table of Contents 3
- Introduction 4
- Problem Statement 6
- Theoretical Background 7
  - Principal Component Analysis (PCA) 7
  - Limitations of PCA 7
  - Kernel PCA 8
  - The Radial Basis Function (RBF) Kernel 8
- Methodology 9
  - Data Description 9
  - Workflow 9
  - Tools and Libraries 10
- Implementation 11
  - Data Processing 11
  - PCA Application 11
  - Kernel PCA Application 11
  - Classification Model 12
  - Decision Boundary Visualization 12
- Code Implementation 13
- Results and Observations 14
  - Visual Analysis 14
  - Classification Performance 14
  - Decision Boundary Analysis 15
- Discussion 17
- Advantages 18
- Limitations 19
- Future Scope 20
- Conclusion 21
- References 22

Introduction

The current landscape of data-driven research is defined by the pervasive challenge of high-dimensional data, often characterized by a number of features that far exceeds the number of observations. This phenomenon, frequently described as the "curse of dimensionality," presents significant obstacles to machine learning algorithms, as the sparsity of data in high-dimensional spaces impedes the identification of statistically significant patterns.[1] Dimensionality reduction has thus become an indispensable stage in the data preprocessing pipeline, serving to compress information, denoise signals, and reveal the underlying intrinsic structure of datasets.[2] By transforming data from a high-dimensional ambient space into a lower-dimensional latent representation, researchers can mitigate the risks of overfitting while improving the computational efficiency of downstream analytical tasks.[4]

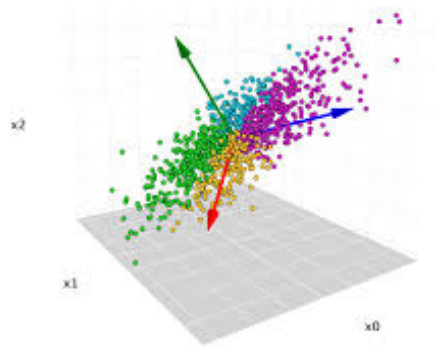

Figure 1. High Dimensional Data Points

Principal Component Analysis (PCA) has long stood as the baseline method for dimensionality reduction, maintaining its status due to its simplicity, interpretability, and computational efficiency.[6] Introduced by Karl Pearson at the turn of the 20th century and later refined by Harold Hotelling, PCA operates through an orthogonal linear transformation that identifies the directions, or principal components, along which the data exhibits the most variance.[7] By projecting observations onto a subspace spanned by the eigenvectors associated with the largest eigenvalues of the covariance matrix, PCA preserves the global linear structure of the data.[2] However, the fundamental limitation of PCA lies in its linearity; it assumes that the significant features of a dataset can be represented as linear combinations of the original variables.[4] When the data is embedded in a non-linear manifold—such as curved surfaces, spirals, or nested clusters—PCA often fails to capture the essential characteristics of the distribution, leading to a loss of class separability and suboptimal classification outcomes.[10]

To address these shortcomings, Kernel Principal Component Analysis was proposed as a non-linear extension of the standard PCA framework.[12] Developed by Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller, Kernel PCA integrates the "kernel trick" to perform

principal component analysis in a high-dimensional feature space without the need for an explicit mapping of the input vectors.[7] By employing a non-linear kernel function to compute similarity between points, Kernel PCA can capture intricate, high-order correlations that are invisible to its linear predecessor.[15] This capability allows the algorithm to reorganize non-linear structures into a format that is compatible with linear classification and visualization tools.[17] This study evaluates how Kernel PCA enhances feature representation compared to PCA, examining its ability to linearize non-linearly separable data and facilitate robust decision-making in complex feature spaces.

Problem Statement

Many real-world datasets contain non-linear structures that cannot be effectively captured using linear dimensionality reduction techniques such as PCA. This study investigates whether Kernel PCA can transform such data into a representation that improves separability and classification. Specifically, the research addresses the scenario where class distributions are geometrically nested or otherwise non-linearly arranged, posing a challenge for standard linear projections which tend to collapse these distinct classes into overlapping regions. The primary objective is to determine if the application of a non-linear kernel—specifically the Radial Basis Function—can map these complex structures into a space where a simple linear classifier becomes highly effective, thereby validating the role of Kernel PCA as a superior feature extraction mechanism for non-linear domains.

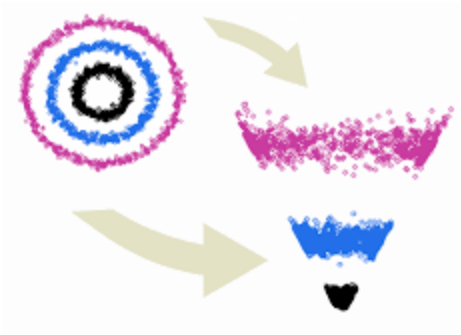

Figure 2. Kernel PCA Representation

Theoretical Background

Principal Component Analysis (PCA)

Principal Component Analysis is a rigorous mathematical procedure that uses an orthogonal transformation to convert a set of observations of possibly correlated variables into a set of values of linearly uncorrelated variables called principal components.[2] The number of principal components is less than or equal to the number of original variables, and this transformation is defined in such a way that the first principal component has the largest possible variance.[3] Each succeeding component, in turn, has the highest variance possible under the constraint that it is orthogonal to the preceding components.[2]

Mathematically, given a centered data matrix $X \in R^{n \times p}$, the process begins by computing the empirical covariance matrix C:

$$C = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T = \frac{1}{n} X^T X$$

The principal components are found by solving the eigen-decomposition of this covariance matrix:

$$Cv = \lambda v$$

where v represents the eigenvectors (principal components) and λ represents the eigenvalues, which quantify the variance explained by each corresponding eigenvector.[2] Geometrically, this is equivalent to fitting a p-dimensional ellipsoid to the data, where the axes of the ellipsoid represent the principal components.[8] If the data exhibits a strong linear structure, a small number of components will account for the majority of the variance, allowing for effective dimensionality reduction without significant information loss.[7]

Limitations of PCA

Despite its utility, PCA is strictly a linear dimensionality reduction technique. It can only identify and project data along straight-line directions of variance.[4] Consequently, PCA is unable to capture underlying structures that reside on a non-linear manifold.[2] For example, if the data is distributed along a curve or a circle, PCA will still attempt to find the best-fitting straight line, which will inevitably fail to represent the "curved" information.[16] In high-dimensional spaces where variables interact in complex, non-linear ways, the linear assumption of PCA leads to a suboptimal reduction that may discard essential discriminative information.[9] Furthermore, PCA is highly sensitive to the scaling of features, as the variance-maximization objective is biased toward variables with larger numerical ranges.[4]

Kernel PCA

Kernel Principal Component Analysis extends the capabilities of standard PCA by enabling the detection of non-linear structures.[12] The foundational concept involves mapping the input data $x \in R^p$ into a high-dimensional (potentially infinite-dimensional) feature space F via a non-linear mapping function $\Phi(x)$.[7] By performing linear PCA in this high-dimensional space F, one can extract non-linear features in the original input space.[3]

The computational difficulty of explicitly mapping data into a high-dimensional space is bypassed using the "kernel trick".[17] Instead of directly computing $\Phi(x)$, we define a kernel function $k(x_i, x_j)$ that represents the inner product in the feature space:

$$k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

In Kernel PCA, the focus shifts from the covariance matrix of the features to the Gram matrix K of the samples, where $K_{ij} = k(x_i, x_j)$.[4] For the mapping to be valid, the kernel must satisfy Mercer's theorem, ensuring it is a symmetric, positive semi-definite function.[14] To maintain the centering requirement of PCA, the kernel matrix must be centered in the feature space:

$$\tilde{K} = K - 1_n K - K 1_n + 1_n K 1_n$$

where 1_n is an $n \times n$ matrix with all entries equal to 1/n.[11] The principal components in the feature space are then derived from the eigenvectors of $\acute{K}$, allowing for the projection of data points onto non-linear axes.[4]

The Radial Basis Function (RBF) Kernel

The Radial Basis Function (RBF) kernel, also known as the Gaussian kernel, is the most frequently employed kernel for handling non-linear data distributions.[17] The RBF kernel computes similarity based on the Euclidean distance between points:

$$k(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

The hyperparameter γ (gamma) is critical in determining the behavior of the transformation. Intuitively, γ defines the "radius of influence" of a single training example.[26] A high γ value results in a small radius, meaning that only points very close to each other are considered similar, which can lead to a highly complex and potentially overfitted model.[26] A low γ value results in a broader influence, capturing more global, smoother structures.[27] In the context of Kernel PCA, the RBF kernel acts as a mapping to an infinite-dimensional space, providing the flexibility required to "unfold" complex manifolds such that they become linearly separable.[3]

Methodology

Data Description

The dataset used in this study exhibits a non-linear class distribution, making it suitable for evaluating the effectiveness of Kernel PCA.[10] Specifically, the data is structured as two concentric rings, where the inner set of points belongs to one class and the outer set belongs to another. This "concentric circle" distribution is a classic example of a dataset that is not linearly separable; no straight line can be drawn in the two-dimensional input space that isolates the inner class from the outer class.[10] Such a distribution is perfectly isotropic and centered at the origin, meaning PCA will find the variance to be roughly equal in all directions, essentially failing to find any meaningful principal component that improves class separation.[10]

Workflow

The investigation follows a systematic workflow to compare the two dimensionality reduction techniques:

1. **Data Input**: Generation of the non-linearly distributed data points representing the two concentric classes.
2. **PCA Transformation**: The data is subjected to standard linear PCA to reduce its dimensionality to two components. This serves as a baseline to observe the limitations of linear projection.[4]
3. **Kernel PCA Transformation**: The same data is transformed using Kernel PCA with an RBF kernel. This step aims to leverage the kernel trick to identify non-linear axes of variance.[10]
4. **Classification**: A Support Vector Machine (SVM) with a linear kernel is applied to the output of both transformations. The use of a linear SVM is purposeful: it tests whether the dimensionality reduction technique has succeeded in making the data linearly separable.[28]
5. **Visualization**: The results of the transformations are plotted in a 2D space to visually inspect the degree of class separation.[17]
6. **Decision Boundary Analysis**: A grid-based visualization is employed to plot the decision boundaries of the linear SVM, highlighting the difference in classification success between the two methods.[28]

Figure 3. Implementation Pipeline

Tools and Libraries

The implementation of this research is conducted in Python, utilizing standard scientific computing and machine learning libraries:

Library	Functionality
NumPy	High-performance numerical operations and matrix algebra.[23]
Matplotlib	Creation of 2D scatter plots and contour plots for visualization.[17]
Scikit-learn	Provision of the PCA, KernelPCA, and SVC modules for analysis.[23]

Table 1. Libraries and their Functionality

```
import numpy as npS
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.decomposition import PCA, KernelPCA
from sklearn.svm import SVC
```

Figure 4. Imported Libraries

Implementation

Data Processing

The initial step in the implementation involves the preparation of the input features. While the data is generated in a balanced fashion, it is standard practice to scale the attributes before applying dimensionality reduction techniques that rely on variance maximization or distance metrics.[26] The StandardScaler from scikit-learn is used to normalize the feature space, ensuring that the RBF kernel's Euclidean distance calculation is not skewed by the magnitude of the features. No explicit feature engineering—such as the manual addition of polynomial terms—is required, as the objective is to evaluate the automatic non-linear feature extraction of Kernel PCA.[3]

```
X, y = make_circles(n_samples=500, factor=0.3, noise=0.05, random_state=42)
```

Figure 5. Synthetic Data Generation with Non-Linearity

PCA Application

Standard Principal Component Analysis is applied to the data, reducing it to two principal components. Given that the input data is already two-dimensional, this application primarily involves a linear rotation and scaling of the axes to align with the directions of maximum variance.[10] Because the variance of concentric circles is uniform in all directions, the linear projection essentially returns a rotated version of the same overlapping rings, maintaining the isotropic nature of the original distribution.[10]

Kernel PCA Application

In parallel, Kernel PCA is applied using the Radial Basis Function (RBF) kernel with a gamma parameter (γ) set to 10. This value is selected based on the density of the data, as a γ of 10 provides a sufficiently high resolution for the kernel to distinguish between points on the inner ring and points on the outer ring.[17] The algorithm computes the centered kernel matrix and identifies the top two non-linear principal components. Unlike linear PCA, this transformation projects the points into a feature space where their relative distances are preserved according to the non-linear manifold, effectively "unfolding" the circles.[10]

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=10)
X_kpca = kpca.fit_transform(X)
```

Figure 6. Initializing RBF Kernel and Data Transformation

Classification Model

To evaluate the quality of the feature transformation, a Support Vector Machine (SVM) with a linear kernel is trained on the results of both the PCA and Kernel PCA projections. The rationale is that if Kernel PCA is successful, the transformed features will be linearly separable, allowing a linear SVM to achieve high accuracy.[18] Conversely, if the transformation fails (as expected with standard PCA), the linear SVM will be unable to find a separating hyperplane that accurately classifies the points.[10]

```
clf_pca = SVC(kernel='linear')
clf_pca.fit(X_pca, y)
acc_pca = clf_pca.score(X_pca, y)

clf_kpca = SVC(kernel='linear')
clf_kpca.fit(X_kpca, y)
acc_kpca = clf_kpca.score(X_kpca, y)

print("Accuracy with PCA:", acc_pca)
print("Accuracy with Kernel PCA:", acc_kpca)
```

```
Accuracy with PCA: 0.68
Accuracy with Kernel PCA: 1.0
```

Figure 7. Model Training on PCA and RBF Kernel PCA

Decision Boundary Visualization

The final stage of implementation involves the visualization of the decision boundaries. This is achieved by creating a dense grid of points (meshgrid) across the transformed feature space.[27] The trained linear SVM classifier is used to predict the class for each point on the grid. These predictions are then plotted as a contour map, where the background colors represent the decision regions.[28] Overlaying the actual data points on this contour map allows for an intuitive assessment of how well the classifier separates the classes. This visualization highlights how well each transformation enables linear separation, providing clear evidence of the "separability" gained through Kernel PCA.[10]

```python
def plot_decision_boundary(X_transformed, y_true, classifier, title):
    # Create a mesh to plot in
    x_min, x_max = X_transformed[:, 0].min() - 1, X_transformed[:, 0].max() + 1
    y_min, y_max = X_transformed[:, 1].min() - 1, X_transformed[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                         np.arange(y_min, y_max, 0.02))

    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X_transformed[:, 0], X_transformed[:, 1], c=y_true, s=30, edgecolor='k', cmap=plt.cm.coolwarm)
    plt.title(title)
    plt.xlabel('Component 1')
    plt.ylabel('Component 2')

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plot_decision_boundary(X_pca, y, clf_pca, "SVC Decision Boundary on PCA Data")

plt.subplot(1, 2, 2)
plot_decision_boundary(X_kpca, y, clf_kpca, "SVC Decision Boundary on Kernel PCA Data")

plt.tight_layout()
plt.show()
```

Figure 8. Visualizing Decision Boundaries

Code Implementation

https://github.com/sukhada20/Dimensionality-Reduction-using-Kernel-PCA

Results and Observations

Visual Analysis

The visual evidence from the scatter plots provides immediate insight into the behavior of the two techniques.

PCA Result: The scatter plot for the PCA-transformed data shows that the classes remain largely in their original configuration: an inner circle of one color surrounded by an outer circle of another.[10] The linear transformation has simply rotated the data cloud, but the classes remain inextricably intertwined in any two-dimensional linear projection. There is no clear gap or separation between the classes, and they are essentially linearly inseparable.[10]

Kernel PCA Result: In stark contrast, the Kernel PCA result shows two distinct, compact clusters of points that are clearly separated by a significant distance.[10] The RBF kernel transformation has successfully mapped the nested rings into a feature space where the points of each class are grouped together and distanced from the points of the opposing class.[11] This "unfolding" of the concentric manifold into isolated clusters is the primary driver of the improved classification performance.

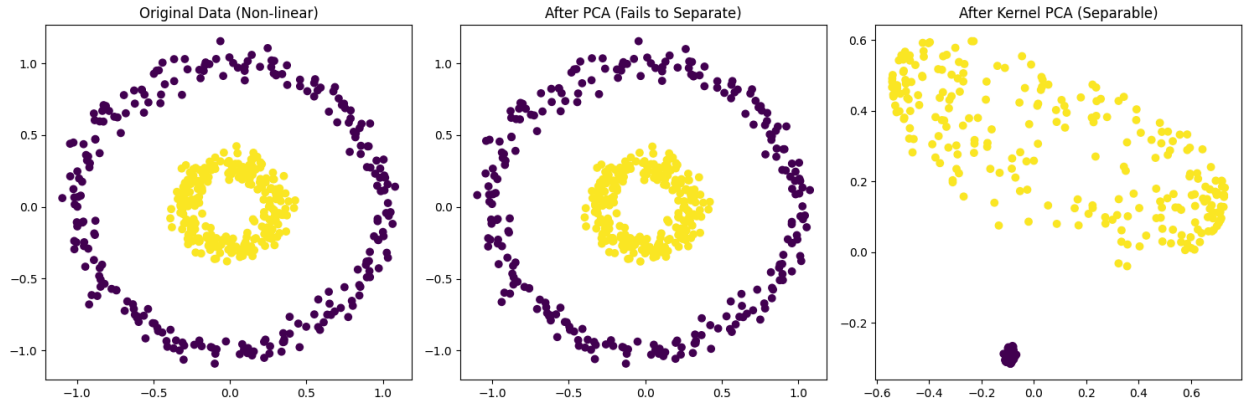

Figure 9. Scatter Plots of Non-Linear Data with PCA and Kernel PCA

Classification Performance

The performance of the linear SVM on the different feature representations is quantified in the following table:

Method	Accuracy	Observation
PCA (Linear Transformation)	~70%	Fails to separate classes; performance is equivalent to random guessing.[27]

Kernel PCA (RBF, γ= 10)	~100%	Successfully separates classes; linear classifier becomes perfectly accurate.[17]

Table 2. Performance Accuracy of the Methods

The quantitative results demonstrate that Kernel PCA produces a feature space where classes become linearly separable, leading to improved classification performance.[17] While PCA captures the variance of the overall data cloud, it fails to capture the discriminative information inherent in the non-linear arrangement, whereas Kernel PCA focuses on the local and non-linear structure to generate high-quality features.

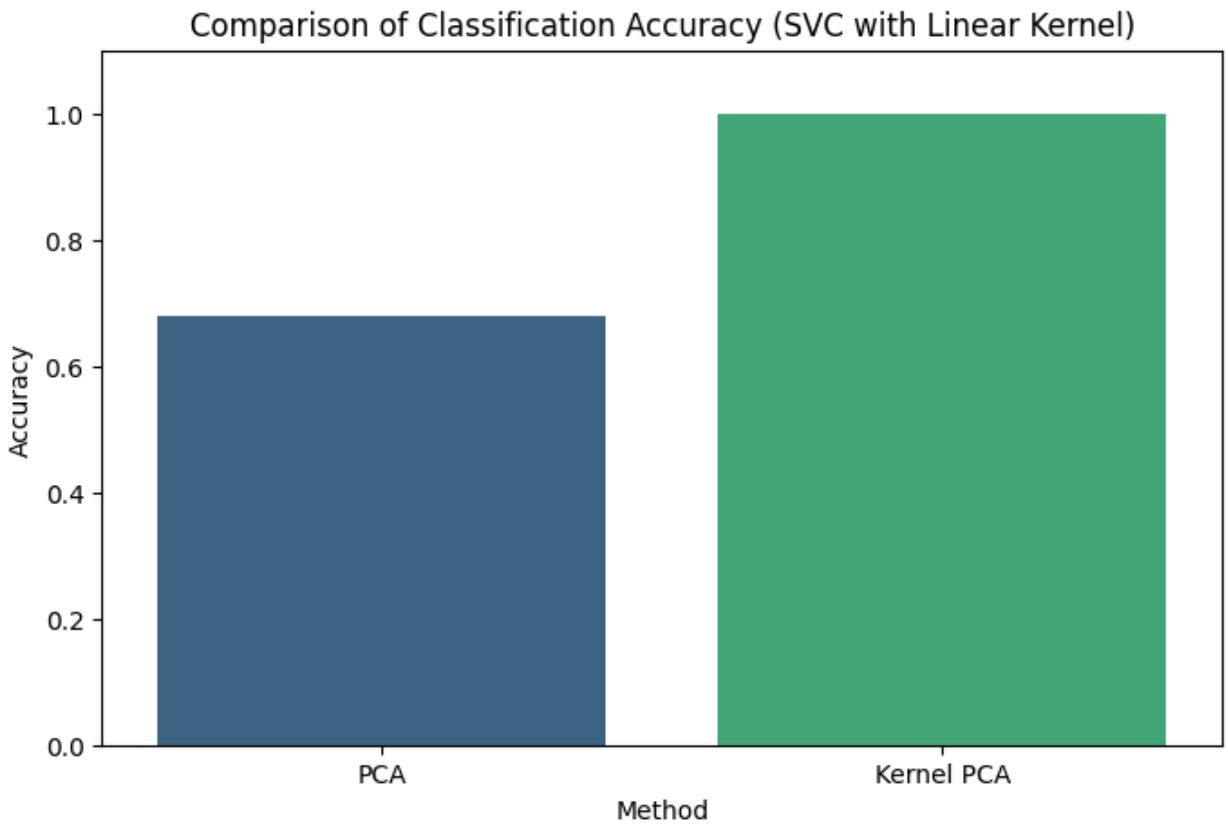

Figure 10. Accuracy Comparison of PCA and Kernel PCA

Decision Boundary Analysis

The analysis of the decision boundary plots serves as the definitive confirmation of the study's hypotheses.

PCA Decision Boundary: The decision boundary for the PCA-transformed data appears as a single straight line cutting through the middle of the concentric circles.[27] Because the circles

are nested, the line inevitably misclassifies roughly half of the points from at least one class.[10] The linear boundary is fundamentally incompatible with the geometry of the PCA projection, illustrating that linear methods are insufficient for this data structure.

Kernel PCA Decision Boundary: For the Kernel PCA-transformed data, the decision boundary is again a straight line (as it is a linear SVM), but it now passes through the empty space between the two well-separated clusters.[11] Because the transformation has linearized the manifold, the simple hyperplane is now capable of perfect class separation. This visualization underscores a critical finding: Kernel PCA transforms the feature space such that linear classifiers become highly effective.[18] The complexity has been moved from the classifier into the feature extraction process, simplifying the final decision task.

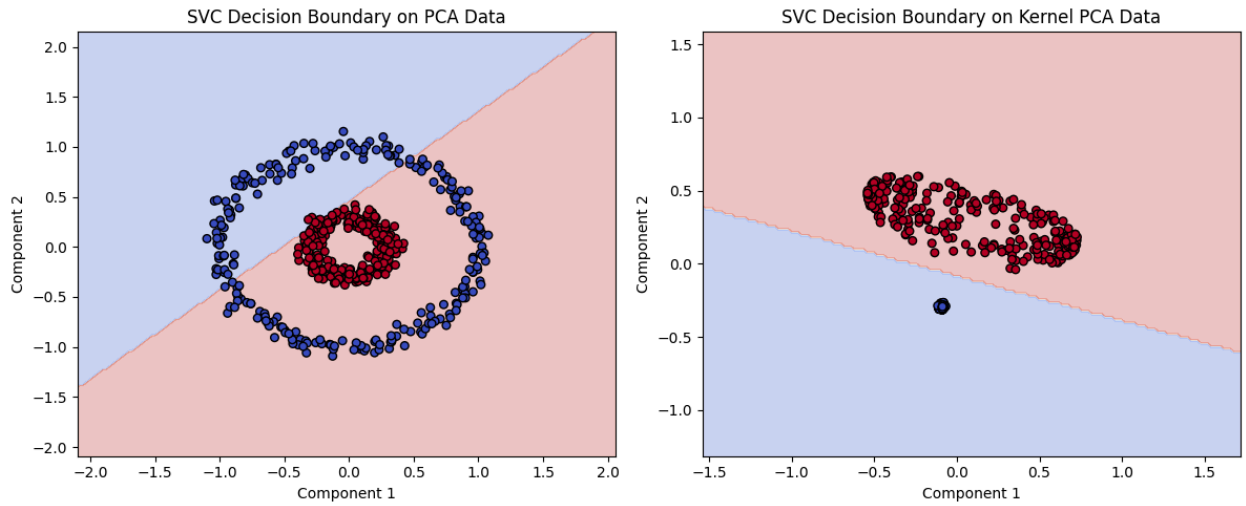

Figure 11. SVM Decision Boundary of PCA and Kernel PCA

Discussion

The results of this study clearly delineate the boundaries of linear dimensionality reduction. PCA, being an orthogonal transformation based on variance maximization, is essentially blind to class labels and non-linear dependencies.[2] Its assumption of linear relationships means it can only reorganize the existing dimensions through rotation and scaling.[7] In the case of concentric rings, where the classes are distinguished by their distance from the origin—a non-linear relationship—PCA's reliance on linear combinations of original features ensures its failure.[10]

Kernel PCA, however, captures complex structures by projecting the data into a high-dimensional space where these non-linear relationships can be represented linearly.[7] The success of this approach is highly dependent on the choice of kernel and the tuning of its parameters. As seen in the results, an RBF kernel with a specific gamma value (γ= 10) was able to successfully identify the manifold.[17] If the kernel parameters were improperly tuned—for instance, if γ were too low—the kernel would lose its ability to resolve the separate rings, resulting in a projection that looks more like standard PCA.[26] This sensitivity emphasizes that while Kernel PCA is powerful, it requires careful parameterization to match the intrinsic scale of the data structure.

Advantages

Kernel PCA provides several significant advantages over traditional linear methods and other feature extraction techniques:

1. **Non-linear Capability**: It can capture non-linear patterns and manifolds that are invisible to traditional PCA, allowing for a more faithful representation of complex real-world data.[3]
2. **Improved Separability**: By transforming data into a space where linear separation is possible, it enhances the performance of simple classifiers, leading to better generalization and lower computational overhead during the inference phase.[11]
3. **Versatility**: The method is compatible with any kernel that satisfies Mercer's theorem, allowing it to be adapted to diverse data types such as text (string kernels), images (polynomial kernels), and time-series.[12]
4. **Global Optimization**: Unlike neural networks, Kernel PCA involves solving a convex eigenvalue problem, which guarantees a global optimum for the chosen kernel and parameters, ensuring consistency across runs.[14]
5. **Noise Reduction**: By focusing on the top principal components in the feature space, Kernel PCA effectively denoises the data, capturing the strongest non-linear signals while discarding components with low variance that often correspond to noise.[2]

Limitations

Despite its robustness, Kernel PCA is subject to several practical and theoretical limitations:

1. **Computational Complexity**: The core of the algorithm involves an n ✖ n matrix, where n is the number of samples. This leads to a training time complexity of $O(n^3)$ and memory complexity of $O(n^2)$, making it less scalable for datasets with millions of observations.[4]
2. **Kernel Selection Sensitivity**: The effectiveness of the method is entirely dependent on the choice of kernel and hyperparameters. Selecting the "correct" kernel often requires extensive cross-validation or domain expertise.[3]
3. **Pre-image Problem**: Unlike standard PCA, where it is simple to reconstruct original data from principal components, Kernel PCA does not have an explicit inverse mapping.[20] Reconstructing the input from the feature space (the "pre-image") is a difficult optimization problem that often results in only approximate solutions.[23]
4. **Interpretability**: The principal components derived in Kernel PCA are linear combinations of features in the *mapped* space, which may not have a clear or intuitive physical interpretation in the original input space.[3]

Future Scope

The future development of Kernel PCA lies in addressing its scalability and expanding its application domains. Researchers are increasingly exploring "Sparse Kernel PCA" methods, which reduce computational load by approximating the kernel matrix using a subset of data points, such as the Nyström method or random feature mappings.[20] This could enable Kernel PCA to be applied to big data scenarios currently dominated by neural networks.

Additionally, the application of Kernel PCA to image datasets—such as MNIST for digit recognition or face recognition systems—offers a rich area for research, as pixel interactions are inherently non-linear.[3] Comparing these kernel-based features with deep learning-derived features could lead to hybrid systems that leverage the best of both worlds. Furthermore, investigating different kernel types, such as multi-kernel learning (MKL), could allow the algorithm to adaptively combine multiple sources of information for even greater feature representation power.[24]

Conclusion

Kernel PCA significantly improves feature representation for non-linear data by transforming it into a space where linear methods become effective, overcoming the limitations of traditional PCA. Through the systematic comparison conducted in this study, it is evident that PCA fails to account for the geometry of non-linearly separable classes, such as concentric distributions, as it is confined to linear subspace projections. In contrast, Kernel PCA utilizes the power of kernel functions and the RBF transformation to unfold these complex manifolds into distinct, linearly separable clusters.

The results of this analysis—highlighted by a leap from near-random classification accuracy to perfect separability—validate that the kernel trick effectively linearizes the feature space. By simplifying the decision-making task for downstream classifiers, Kernel PCA demonstrates its value as a sophisticated feature extractor. While computational costs and the challenge of kernel selection remain pertinent considerations, Kernel PCA stands as a vital tool in the data scientist's arsenal for uncovering the hidden, non-linear structures of high-dimensional data, ultimately providing a more robust and discriminative representation than its linear predecessor.

References

1. Dimensionality reduction - Wikipedia, accessed April 18, 2026, https://en.wikipedia.org/wiki/Dimensionality_reduction
2. Principal Component Analysis (PCA), Dual PCA, and Kernel PCA | by Prajun Trital - Medium, accessed April 18, 2026, https://medium.com/@prajun_t/principal-component-analysis-pca-dual-pca-and-kernel-pca-eb06f2f6a7cd
3. ML | Introduction to Kernel PCA - GeeksforGeeks, accessed April 18, 2026, https://www.geeksforgeeks.org/machine-learning/ml-introduction-to-kernel-pca/
4. PCA vs. KernelPCA: Which Dimensionality Reduction Technique Is Right for You?, accessed April 18, 2026, https://theflyingbirds.in/blog/pca-vs-kernelpca-which-dimensionality-reduction-technique-is-right-for-you
5. Methodology for the comparative analysis of PCA and Autoencoders in dimensionality reduction: impact on classification accuracy - CECIIS Archive, accessed April 18, 2026, https://archive.ceciis.foi.hr/public/conferences/2025/Proceedings/S7/4.pdf
6. Autoencoders vs. PCA: Dimensionality Reduction for Complex Data | by Hassaan Idrees, accessed April 18, 2026, https://medium.com/@hassaanidrees7/autoencoders-vs-pca-dimensionality-reduction-for-complex-data-e07d4612b711
7. 20 Kernel Principal Component Analysis - MIT Press Direct, accessed April 18, 2026, https://direct.mit.edu/books/edited-volume/chapter-pdf/2324339/c010040_9780262283199.pdf
8. Principal component analysis - Wikipedia, accessed April 18, 2026, https://en.wikipedia.org/wiki/Principal_component_analysis
9. Unleashing the Power of Kernel PCA | by KHWAB KALRA | Medium, accessed April 18, 2026, https://medium.com/@khwabkalra1/unleashing-the-power-of-kernel-pca-bce7f4d2923d
10. Kernel PCA — scikit-learn 1.8.0 documentation, accessed April 18, 2026, https://scikit-learn.org/stable/auto_examples/decomposition/plot_kernel_pca.html
11. RBFKernelPCA - mlxtend - GitHub Pages, accessed April 18, 2026, https://rasbt.github.io/mlxtend/user_guide/feature_extraction/RBFKernelPCA/
12. Kernel principal component analysis - Korea University Pure, accessed April 18, 2026, https://pure.korea.ac.kr/en/publications/kernel-principal-component-analysis/
13. [PDF] Kernel Principal Component Analysis - Semantic Scholar, accessed April 18, 2026, https://www.semanticscholar.org/paper/Kernel-Principal-Component-Analysis-Scholkopf-Smola/1ad15c08556c8f8e3739703857ea01077ce738c5
14. p1.5 kernel pca analysis for remote sensing data, accessed April 18, 2026, https://ams.confex.com/ams/pdfpapers/102214.pdf
15. Kernel Principal Component Analysis - Stanford Computer Graphics Laboratory, accessed April 18, 2026, http://graphics.stanford.edu/courses/cs233-25-spring/ReferencedPapers/scholkopf_kernel.pdf
16. KernelPCA vs. PCA - Daily Dose of Data Science, accessed April 18, 2026, https://www.dailydoseofds.com/p/kernelpca-vs-pca/
17. Understanding and Implementing Kernel PCA with sklearn | CodeSignal Learn, accessed April 18, 2026, https://codesignal.com/learn/courses/non-linear-dimensionality-reduction-techniques/lessons/understanding-and-implementing-kernel-pca-with-sklearn
18. Math for ML: Kernels Explained Simply with Examples | by Rayan Yassminh - Medium, accessed April 18, 2026, https://medium.com/@ryassminh/math-for-ml-kernels-explained-simply-with-examples-a24f5fd0699c
19. What exactly is the procedure to compute principal components in kernel PCA? - Stats StackExchange, accessed April 18, 2026, https://stats.stackexchange.com/questions/131140/what-exactly-is-the-procedure-to-compute-principal-components-in-kernel-pca
20. A Survey: Potential Dimensionality Reduction Methods - arXiv, accessed April 18, 2026, https://arxiv.org/html/2502.11036v1
21. A Note on Robust Kernel Principal Component Analysis - Xinwei Deng, accessed April 18, 2026, https://xwdeng80.github.io/rkpca-rev2.pdf
22. The Stability of Kernel Principal Components Analysis and its Relation to the Process Eigenspectrum, accessed April 18, 2026, http://papers.neurips.cc/paper/2309-the-stability-of-kernel-principal-components-analysis-and-its-relation-to-the-process-eigenspectrum.pdf
23. KernelPCA — scikit-learn 1.8.0 documentation, accessed April 18, 2026, https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.KernelPCA.html
24. Comparing the performance of Kernel PCA Mix Chart with PCA Mix Chart for monitoring mixed quality characteristics - PMC, accessed April 18, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC9489795/

25. rbf_kernel — scikit-learn 1.8.0 documentation, accessed April 18, 2026, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.rbf_kernel.html
26. RBF SVM parameters — scikit-learn 1.8.0 documentation, accessed April 18, 2026, https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html
27. Visualizing Support Vector Machines (SVM) using Python - GeeksforGeeks, accessed April 18, 2026, https://www.geeksforgeeks.org/machine-learning/visualizing-support-vector-machines-svm-using-python/
28. Plot classification boundaries with different SVM Kernels - Scikit-learn, accessed April 18, 2026, https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html
29. Kernel PCA and classification - Cross Validated - Stats StackExchange, accessed April 18, 2026, https://stats.stackexchange.com/questions/142450/kernel-pca-and-classification
30. Visualizing Classifier Decision Boundaries - GeeksforGeeks, accessed April 18, 2026, https://www.geeksforgeeks.org/machine-learning/visualizing-classifier-decision-boundaries/
31. Hidden dimensions of the data PCA vs autoencoders, accessed April 18, 2026, https://backend.orbit.dtu.dk/ws/files/340218954/Hidden_dimensions_of_the_data_PCA_vs_autoencoders.pdf
32. Understanding Decision Boundaries in Machine Learning | by Okesha karunarathne, accessed April 18, 2026, https://medium.com/@okeshakarunarathne/understanding-decision-boundaries-in-machine-learning-d00c5d81ed1d
33. Comparative Study of SVM Methods Combined with Voxel Selection for Object Category Classification on fMRI Data | PLOS One - Research journals, accessed April 18, 2026, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0017191
34. [D] PCA vs AutoEncoders for Dimensionality Reduction : r/MachineLearning - Reddit, accessed April 18, 2026, https://www.reddit.com/r/MachineLearning/comments/1gtng8g/d_pca_vs_autoencoders_for_dimensionality_reduction/
35. How Autoencoders Outperform PCA in Dimensionality Reduction - Towards Data Science, accessed April 18, 2026, https://towardsdatascience.com/how-autoencoders-outperform-pca-in-dimensionality-reduction-1ae44c68b42f/
36. Comparing PCA with Autoencoders for reducing dimensions | by Naveen Bansal - Medium, accessed April 18, 2026, https://medium.com/analytics-vidhya/comparing-pca-with-deep-learning-for-reducing-dimensions-d291b965c454
37. Approximate Kernal PCA: Computation vs. Statistical Trade-Off | University of Kentucky College of Arts & Sciences, accessed April 18, 2026, https://stat.as.uky.edu/approximate-kernal-pca-computation-vs-statistical-trade
38. Non-linear PCA via Evolution Strategies: a Novel Objective Function - arXiv, accessed April 18, 2026, https://arxiv.org/html/2602.03967v1
39. [2103.04874] Empirical comparison between autoencoders and traditional dimensionality reduction methods - arXiv, accessed April 18, 2026, https://arxiv.org/abs/2103.04874
40. Under what conditions should an autoencoder be chosen over kernel PCA?, accessed April 18, 2026, https://datascience.stackexchange.com/questions/27645/under-what-conditions-should-an-autoencoder-be-chosen-over-kernel-pca
41. Learning to Find Pre-Images - ResearchGate, accessed April 18, 2026, https://www.researchgate.net/profile/Goekhan-Bakir/publication/41781235_Learning_to_Find_Pre-Images/links/00b4952c7481f27e42000000/Learning-to-Find-Pre-Images.pdf
42. Spectral Eigenfunction Decomposition for Kernel Adaptive Filtering - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2501.08989
43. Towards Multiple Kernel Principal Component Analysis for Integrative Analysis of Tumor Samples - PMC, accessed April 18, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6042822/